

PostgreSQL 16 — DR Test & Revert Runbook

Streaming Replication: Promote standby for DR testing, then rebuild & reattach

Exercise window	Day 1 • Mon Jun 22 (08:00--09:30 EST) Day 2 • Tue Jun 23 (17:00--18:00 EST)
Engine	PostgreSQL 16 • physical streaming replication (with WAL archiving)
Production primary	db01 — stays primary & online throughout (no downtime)
Standby	db02 — promoted Day 1, rebuilt & demoted to read-only Day 2
Rebuild methods	Option A • pg_rewind Option B • pg_basebackup
Author / role	Lead DBA (Technical) — SS&C Private Cloud

0 Environment, assumptions & conventions

The runbook is written against your standard environment. Values marked **CONFIRM** are assumptions taken from the email screenshot and your standing setup — verify (or correct) them before execution. Each appears as a placeholder like `<PRIMARY_FQDN>` in the commands.

Item	Value used in this runbook	Status
Data directory (PGDATA)	<code>/apps/pgsql_data/16</code>	standard
WAL archive	<code>/apps/pgsql_archives/</code>	standard
Server logs	<code>/apps/logs</code>	standard
Backup images	<code>/apps/backups</code>	standard
Primary FQDN <code><PRIMARY_FQDN></code>	<code>ars-acm-nibc-...-db01.ssn-corp.cloud</code> (partly cut off in image)	CONFIRM
Standby FQDN	<code>ars-acm-nibc-ams-postgres-db02.ssn-corp.cloud</code>	from image
Listen port	<code>5432</code>	CONFIRM
Replication role <code><REPL_USER></code>	<code>replicator</code>	confirmed
Physical slot <code><SL0T></code>	<code>ssncpri</code>	confirmed
PG binary path	<code>/usr/pgsql-16/bin</code> (PGDG / AlmaLinux default)	confirmed
pgBackRest stanza	<code><STANZA></code>	CONFIRM

Role mapping — read this first

Throughout: **db01** = the production primary. It is **never demoted or stopped**; it keeps serving production for the entire exercise. **db02** = the standby. On Day 1 it is promoted to a temporary, isolated read-write instance for the app team. On Day 2 it is rebuilt from db01 and put back to read-only standby. For `pg_rewind` on Day 2, the **source** is db01 and the **target** (the server being rewound) is db02.

1 Executive summary & downtime statement

Day 1 promotes db02 so the application team can run a DR test against a live read-write copy, while production continues uninterrupted on db01. Because both servers accept independent writes during the test, they diverge on separate timelines. Day 2 therefore rebuilds db02 from db01 (discarding the test-only writes on db02) and reattaches it as a read-only streaming standby.

Downtime on the primary — the email's closing question

No downtime is required on the primary (db01). db01 is not stopped, restarted, or demoted at any point. The only primary-side actions are (a) dropping the db02 replication slot on Day 1 to stop WAL retention, and (b) recreating it on Day 2 — both are online operations.

2 Global pre-requisites & safety rules

- Take a fresh, verified backup of db01 **before** Day 1 and again before Day 2 (pgBackRest preferred).
- Run every destructive command from the correct host. Confirm hostname and `$PGDATA` first.
- Keep the pre-rebuild copy of db02's `PGDATA` until replication is verified healthy — do not delete it on Day 2 until validation passes.
- For `pg_rewind`: it modifies the target data directory **in place**. Snapshot or copy db02's `PGDATA` first, or be ready to fall back to a full `pg_basebackup`.
- Always run `pg_rewind` with `--dry-run` first.

Hard rule — never reset all on the standby

Do **not** run `ALTER SYSTEM RESET ALL;` on db02. It wipes `primary_conninfo` (and `primary_slot_name`) from `postgresql.auto.conf` and silently breaks replication after the next restart. If you must clear a single parameter, use `ALTER SYSTEM RESET <param>;` for that one key only.

DAY 1 — Prepare the DR environment

Mon, Jun 22 · 08:00--09:30 EST · est. 1.0–1.5 h. Goal: db02 promoted to read-write for app-team DR testing; db01 untouched as production.

1.0 Pre-flight backup & health check

On **db01 (primary)** — confirm replication is healthy and capture baseline state:

```
-- Run as a superuser on db01
SELECT pg_is_in_recovery();                                -- expect: f (db01 is
primary)

SELECT slot_name, slot_type, active, restart_lsn           -- note the db02 slot name
FROM pg_replication_slots;

SELECT application_name, client_addr, state,
       sent_lsn, replay_lsn,
       pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS replay_lag_bytes
FROM pg_stat_replication;                                -- expect state=streaming,
lag ~0

SELECT timeline_id, checkpoint_lsn FROM pg_control_checkpoint();
```

On **db02 (standby)** — confirm it is caught up:

```
SELECT pg_is_in_recovery();                                -- expect: t (db02 is
standby)

SELECT status, sender_host, sender_port, received_lsn, latest_end_lsn
FROM pg_stat_wal_receiver;

SELECT pg_last_wal_receive_lsn(), pg_last_wal_replay_lsn(),
       now() - pg_last_xact_replay_timestamp() AS replay_delay;
```

Safety backup of db01 (choose one — pgBackRest preferred):

```
# Preferred: pgBackRest full backup
pgbackrest --stanza=<STANZA> --type=full backup
pgbackrest --stanza=<STANZA> info                          # verify backup is listed/
valid

# Alternative: compressed base backup snapshot of db01
/usr/pgsql-16/bin/pg_basebackup -h <PRIMARY_FQDN> -p 5432 -U replicator \
-D /apps/backups/db01_preDR_$(date +%Y%m%d) -Ft -z -Xs -P
```

1.1 Confirm db02 is fully synchronized (final cut)

```
-- On db01: force a checkpoint and read the current write position
CHECKPOINT;
SELECT pg_current_wal_lsn() AS primary_lsn;
```

```
-- On db02: confirm replay has reached (>=) the primary_lsn above
SELECT pg_last_wal_replay_lsn() AS standby_replayed;
```

Proceed only when `standby_replayed` is at or beyond `primary_lsn`. A perfectly clean cut (briefly pausing app writes on db01) is optional here, since db02 is rebuilt on Day 2 regardless.

1.2 Promote the standby (db02)

PostgreSQL 16 promotion method

In PG16 the old `promote_trigger_file` / trigger-file mechanism has been **removed**. Promote with `pg_ctl promote` or the SQL function `pg_promote()` — not a trigger file.

```
# On db02 host, as the postgres OS user
/usr/pgsql-16/bin/pg_ctl -D /apps/pgsql_data/16 promote
```

...or equivalently from a psql session on db02:

```
SELECT pg_promote(wait => true, wait_seconds => 60);           -- returns t on success
```

Verify the promotion on db02:

```
SELECT pg_is_in_recovery();                                     -- expect: f (now read-
write)
SELECT timeline_id FROM pg_control_checkpoint();              -- incremented by 1 vs. step
1.0
```

Tail the log under `/apps/logs` and confirm lines similar to “selected new timeline ID” and “database system is ready to accept connections”.

1.3 “Disable replication on the primary” (db01) — release the slot

db02 has disconnected, so its physical slot on db01 is now `active = f` and is pinning WAL. Left alone it will grow `pg_wal` on production for the whole test window. Drop it (it is recreated on Day 2):

```
-- On db01: inspect retained WAL, then drop the inactive slot
SELECT slot_name, active, restart_lsn,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS wal_retained
FROM   pg_replication_slots;

SELECT pg_drop_replication_slot('ssncpri');
```

If you prefer to keep the slot

You may leave the slot in place instead of dropping it — but then you **must monitor free space in `pg_wal` on db01** closely for the duration of the test, because the inactive slot will keep accumulating WAL. Dropping is cleaner here since db02 gets a full rebuild on Day 2.

Optional (stricter): comment the db02 `replication` line in db01's `/apps/pgsql_data/16/pg_hba.conf` and reload — not required.

```
SELECT pg_reload_conf();  
pg_hba.conf
```

```
-- only if you edited
```

1.4 Hand off to the application team

db02 is now an isolated read-write instance. The app team points their DR test at `ars-acm-nibc-ams-postgres-db02.ssn-corp.cloud:5432`. db01 continues serving production.

1.5 Day 1 rollback (if promotion was done in error)

There is no “un-promote.” If db02 was promoted by mistake and the app team has not yet written to it, the correct recovery is simply to run the **Day 2 rebuild early** (Section 4, Option A or B) to re-seed db02 from db01 and reattach it as a standby. db01 is unaffected.

DAY 2 — Revert the DR environment

Tue, Jun 23 · 17:00--18:00 EST · ~1 h excluding resync. Goal: db02 rebuilt from db01 and back to read-only standby; replication healthy.

2.0 Pre-flight

- Confirm the application team has finished testing and disconnected from db02.
- Take a fresh pgBackRest backup of db01 (its data is the source of truth for the rebuild).
- Choose the rebuild method using the gate below.

Choosing the rebuild method

Option A — pg_rewind: fast; copies only blocks that changed since divergence. **Requires** that db02 has `wal_log_hints = on` or data checksums enabled (verify in step 2.3). **Option B — pg_basebackup:** full, guaranteed-clean reseed; always works; use it if the pg_rewind prerequisite is not met, or whenever you want a from-scratch copy.

2.1 Enable replication on the primary (db01)

Recreate the physical slot dropped on Day 1 and confirm replication settings are intact:

```
-- On db01
SELECT pg_create_physical_replication_slot('ssncpri');
SELECT slot_name, active FROM pg_replication_slots;           -- slot present, active=f
(until db02 connects)

SHOW wal_level;        -- expect: replica (or logical)
SHOW max_wal_senders;  -- must exceed the number of standbys
SHOW archive_mode;     -- expect: on (archiving to /apps/pgsql_archives/)
```

If you commented out the db02 line in `pg_hba.conf` during step 1.3, restore it now and `SELECT pg_reload_conf();`.

Skip manual slot creation if pg_basebackup will create it

If you go with Option B and want `pg_basebackup` to create the slot itself (via `-C -S`), **skip** the `pg_create_physical_replication_slot` call above — otherwise basebackup will error that the slot already exists.

2.2 Cleanly shut down db02

A clean shutdown is **mandatory** for `pg_rewind` and tidy for basebackup:

```
/usr/pgsql-16/bin/pg_ctl -D /apps/pgsql_data/16 -m fast stop

# Confirm a clean stop and capture key facts for the next step
/usr/pgsql-16/bin/pg_controldata /apps/pgsql_data/16 \
| grep -E "Database cluster state|checksum|Latest checkpoint location"
# 'Database cluster state' must read: shut down
```

2.3 Option A — rebuild with pg_rewind

Prerequisite gate — run on db02; pg_rewind needs at least one of these to be true:

```
# (1) Data checksums? Look for a non-zero checksum version.
/usr/pgsql-16/bin/pg_controldata /apps/pgsql_data/16 | grep -i "checksum"
# "Data page checksum version: 1" -> checksums ON

# (2) Hint-bit logging?
grep -Ei '^s*wal_log_hints' /apps/pgsql_data/16/postgresql.conf \
                           /apps/pgsql_data/16/postgresql.auto.conf
# "wal_log_hints = on"

# If BOTH are off -> pg_rewind CANNOT run. Switch to Option B (pg_basebackup).
```

On db01 (source), provide a role for pg_rewind. A superuser works; otherwise grant the minimal set:

```
-- On db01 (least-privilege rewind role)
CREATE ROLE rewind_user WITH LOGIN PASSWORD '<secret>';
GRANT EXECUTE ON FUNCTION pg_catalog.pg_ls_dir(text, boolean, boolean)      TO
rewind_user;
GRANT EXECUTE ON FUNCTION pg_catalog.pg_stat_file(text, boolean)            TO
rewind_user;
GRANT EXECUTE ON FUNCTION pg_catalog.pg_read_binary_file(text)              TO
rewind_user;
GRANT EXECUTE ON FUNCTION pg_catalog.pg_read_binary_file(text, bigint, bigint, boolean) TO
rewind_user;
```

Ensure db01's `pg_hba.conf` allows `rewind_user` from db02, and put the password in db02's `~postgres/.pgpass` (mode 0600). Then on **db02 (target)**:

```
# 1) DRY RUN first -- reports what it would do, changes nothing
/usr/pgsql-16/bin/pg_rewind \
--target-pgdata=/apps/pgsql_data/16 \
--source-server="host=<PRIMARY_FQDN> port=5432 user=rewind_user dbname=postgres" \
--restore-target-wal \
--dry-run --progress

# 2) REAL run (only after a clean dry run)
/usr/pgsql-16/bin/pg_rewind \
--target-pgdata=/apps/pgsql_data/16 \
--source-server="host=<PRIMARY_FQDN> port=5432 user=rewind_user dbname=postgres" \
--restore-target-wal \
--write-recovery-conf \
--progress
```

- `--restore-target-wal` lets `pg_rewind` pull pre-divergence WAL from the archive via db02's `restore_command` if those segments are no longer in `pg_wal`.
- `--write-recovery-conf` writes `primary_conninfo` into `postgresql.auto.conf` and creates `standby.signal` automatically.

Then confirm/adjust db02's recovery configuration in `/apps/pgsql_data/16/postgresql.auto.conf`:


```
primary_conninfo = 'host=<PRIMARY_FQDN> port=5432 user=replicator application_name=db02'
primary_slot_name = 'ssncpri'
restore_command = 'cp /apps/pgsql_archives/%f %p'
```

Confirm `standby.signal` exists in `PGDATA` and `hot_standby = on`, then start db02:

```
ls -l /apps/pgsql_data/16/standby.signal
/usr/pgsql-16/bin/pg_ctl -D /apps/pgsql_data/16 -l /apps/logs/postgresql.log start
```

2.4 Option B — rebuild with pg_basebackup

Move the old data directory aside (keep it until verified), then reseed from db01:

```
# Ensure db02 is stopped (step 2.2), then preserve the old PGDATA
mv /apps/pgsql_data/16 /apps/pgsql_data/16.drtest_$(date +%Y%m%d_%H%M)
install -d -m 0700 -o postgres -g postgres /apps/pgsql_data/16

# Full base backup from db01, streaming WAL, writing standby config.
# Slot was recreated in 2.1 -> reference it with -S (do NOT add -C, which recreates it).
/usr/pgsql-16/bin/pg_basebackup \
  -h <PRIMARY_FQDN> -p 5432 -U replicator \
  -D /apps/pgsql_data/16 \
  -Fp -Xs -R -P -v \
  -S ssncpri
```

Flag	Meaning
<code>-Fp</code>	plain format (writes a ready-to-run data directory)
<code>-Xs</code>	stream WAL during the backup (avoids needing extra WAL retention)
<code>-R</code>	write <code>standby.signal</code> + <code>primary_conninfo</code> automatically
<code>-S <slot></code>	use the existing physical slot
<code>-C -S <slot></code>	(alternative) create the slot as part of the backup — only if you did NOT create it in 2.1

Verify db02's `/apps/pgsql_data/16/postgresql.auto.conf` (add `primary_slot_name` / `restore_command` if `-R` did not include them):

```
primary_conninfo = 'host=<PRIMARY_FQDN> port=5432 user=replicator application_name=db02'
primary_slot_name = 'ssncpri'
restore_command = 'cp /apps/pgsql_archives/%f %p'
```

Confirm `hot_standby = on` and `standby.signal` is present, then start db02:

```
/usr/pgsql-16/bin/pg_ctl -D /apps/pgsql_data/16 -l /apps/logs/postgresql.log start
```

2.5 Set the standby back to read-only (verify)

Read-only is automatic once db02 runs with `standby.signal` present and `hot_standby = on`. Prove it:

```
-- On db02
SELECT pg_is_in_recovery();           -- expect: t (read-only standby)
```

```
-- Negative test: a write must be rejected
CREATE TABLE dr_write_test (x int);
-- expect ERROR: cannot execute CREATE TABLE in a read-only transaction
```

2.6 Validate replication end-to-end

```
-- On db01 (primary): db02 should appear and be streaming
SELECT application_name, client_addr, state, sync_state,
       sent_lsn, replay_lsn,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS send_replay_gap
FROM pg_stat_replication;

SELECT slot_name, active, restart_lsn FROM pg_replication_slots; -- active=t
```

```
-- On db02 (standby): receiver connected, lag trending to zero
SELECT status, sender_host, received_lsn, latest_end_lsn, last_msg_receipt_time
FROM pg_stat_wal_receiver;

SELECT now() - pg_last_xact_replay_timestamp() AS replay_delay;
SELECT timeline_id FROM pg_control_checkpoint(); -- must match db01's current timeline
```

Functional confirmation — write a sentinel on db01 and read it on db02:

```
-- db01:
CREATE TABLE IF NOT EXISTS public.repl_check(id int, ts timestamp default now());
INSERT INTO public.repl_check(id) VALUES (1);

-- db02 (after a moment): the row should be present
SELECT * FROM public.repl_check;
```

2.7 Day 2 contingencies

- **pg_rewind prerequisite not met** (no checksums and `wal_log_hints=off`): use Option B. This is common — check it during 2.0 so there are no surprises in the window.
- **pg_rewind fails midway**: it edits `PGDATA` in place, so the target may be inconsistent. Restore the pre-rewind snapshot of db02's `PGDATA`, or proceed with a full `pg_basebackup` (Option B).
- **db02 will not catch up** (“requested WAL segment ... has already been removed”): ensure `restore_command` points at `/apps/pgsql_archives/` and the segments exist there; consider raising `wal_keep_size` on db01 for future tests.
- **Slot already exists** on basebackup: you both pre-created it (2.1) and passed `-C`. Drop `-C`.

3 Failure scenarios & troubleshooting

Symptom → likely cause → resolution, grouped by day, followed by detailed playbooks for the high-risk failures.

3.1 Day 1 quick reference

Symptom / message	Likely cause	Resolution
Replay lag on db02 will not reach ~0 before promotion	walreceiver disconnected, network, or heavy write load on db01	Inspect <code>pg_stat_wal_receiver</code> (db02) and <code>pg_stat_replication</code> (db01); verify connectivity; wait or briefly quiesce app writes. Do not promote until caught up (or accept the point-in-time, since db02 is rebuilt Day 2 anyway).
<code>pg_promote()</code> returns <code>f</code> / promotion appears stuck	Already promoted, recovery paused, or no permission	Check <code>pg_is_in_recovery()</code> ; if paused run <code>SELECT pg_wal_replay_resume();</code> ; review <code>/apps/logs</code> ; fall back to <code>pg_ctl promote</code> .
ERROR: replication slot "ssncpri" is active when dropping	db02's walsender is still connected to db01	Confirm db02 was promoted (it should have disconnected). If a backend lingers, terminate it — see playbook 3.3-D.
<code>pg_wal</code> on db01 growing rapidly during the test	Inactive slot still pinning WAL	Drop <code>ssncpri</code> (step 1.3); or cap with <code>max_slot_wal_keep_size</code> ; watch <code>df</code> on <code>/apps</code> .
App cannot connect to db02 after promotion	<code>pg_hba.conf</code> , <code>listen_addresses</code> , or wrong port	Confirm db02 listens on 5432 and <code>pg_hba.conf</code> permits the app host; verify read-write with <code>pg_is_in_recovery()=f</code> .

3.2 Day 2 quick reference

Symptom / message	Likely cause	Resolution
target server needs to use either data checksums or <code>wal_log_hints = on</code>	<code>pg_rewind</code> prerequisite not met on db02	<code>pg_rewind</code> cannot run — switch to Option B (<code>pg_basebackup</code>). To enable for future tests, set <code>wal_log_hints = on</code> (needs a restart) or initialize with checksums.
could not find common ancestor / no rewind required	db02 did not actually diverge, or unexpected timeline	Confirm db02's timeline advanced at promotion (Day 1 step 1.2). If genuinely no divergence, you can reattach without rewind; otherwise inspect timeline history files in <code>pg_wal</code> .
target server must be shut down cleanly	db02 crashed or was stopped in immediate mode	Start db02, then <code>pg_ctl stop -m fast</code> for a clean checkpoint; re-check <code>pg_controldata</code> reads <i>shut down</i> .
WAL segment 0000... has already been removed (rewind or catch-up)	Required WAL no longer in <code>pg_wal</code>	Set a working <code>restore_command</code> to <code>/apps/pgsql_archives/</code> ; for <code>pg_rewind</code> add <code>--restore-target-wal</code> ; confirm the segment exists in the archive — see playbook 3.3-C.

pg_rewind password authentication failed / permission denied for function	rewind role auth or missing function grants	Fix .pgpass / pg_hba.conf for rewind_user ; apply the four GRANT EXECUTE statements (step 2.3) or use a superuser.
pg_basebackup: replication slot "ssncpri" already exists	Passed -C while the slot was pre-created in 2.1	Drop -C ; use -S ssncpri alone.
pg_basebackup: directory "/apps/pgsql_data/16" exists and is not empty	Old PGDATA not cleared	Move it aside (step 2.4) before re-running.
pg_basebackup: could not create directory / permission denied	Ownership or mode on the new PGDATA	install -d -m 0700 -o postgres -g postgres /apps/pgsql_data/16 ; run as the postgres user.
number of requested standby connections exceeds max_wal_senders	Too few WAL senders on db01	Raise max_wal_senders on db01 (needs restart) or free an idle sender.
db02 starts but FATAL: could not connect to the primary server	Wrong primary_conninfo , missing replication line in db01 pg_hba.conf , or bad password	See playbook 3.3-B.
db02 came up read-write (pg_is_in_recovery()=f)	standby.signal missing → started standalone / self-promoted onto a new timeline	Critical — see playbook 3.3-A.
Slot ssncpri stays active=f after db02 starts	db02 not referencing the slot	Confirm primary_slot_name = 'ssncpri' in db02's postgresql.auto.conf ; restart db02.
Replication healthy but replay lag keeps climbing	Heavy write load, standby disk I/O, or recovery conflicts	Check I/O on db02, max_standby_streaming_delay , and consider hot_standby_feedback=on ; ensure adequate resources.
Accidentally ran ALTER SYSTEM RESET ALL on db02	primary_conninfo + primary_slot_name wiped from postgresql.auto.conf	Re-add both with ALTER SYSTEM SET , confirm standby.signal , restart. Reinforces the hard rule in Section 2.

3.3 Detailed playbooks

A — db02 came up read-write (missing standby.signal / self-promotion)

The most dangerous Day 2 failure: if db02 starts without standby.signal , it boots as a standalone primary, bumps its own timeline, and may accept writes. Recover immediately:

```
# On db02: confirm the symptom
psql -tAc "SELECT pg_is_in_recovery();"      # f => it self-promoted (standby.signal was absent)
```

```
# Stop it cleanly and restore standby identity
/usr/pgsql-16/bin/pg_ctl -D /apps/pgsql_data/16 -m fast stop
ls -l /apps/pgsql_data/16/standby.signal      # if missing, recreate it
touch /apps/pgsql_data/16/standby.signal
```

```
# Ensure these exist in /apps/pgsql_data/16/postgresql.auto.conf
primary_conninfo = 'host=<PRIMARY_FQDN> port=5432 user=replicator application_name=db02'
primary_slot_name = 'ssncpri'
```

Restart db02. If it had already generated WAL on its own timeline, do not trust the in-place fix — re-run a full `pg_basebackup` (Option B) so db02 is a clean copy of db01.

B — db02 cannot connect to the primary

Test the replication connection directly from db02, then fix the failing layer:

```
# From db02 -- a successful IDENTIFY_SYSTEM proves conninfo + auth + hba are all good
psql "host=<PRIMARY_FQDN> port=5432 user=replicator dbname=postgres replication=database" \
-c "IDENTIFY_SYSTEM;"
```

On db01, ensure a replication line exists for db02 in `/apps/pgsql_data/16/pg_hba.conf` and reload:

```
# TYPE      DATABASE      USER      ADDRESS      METHOD
host        replication  replicator <db02-ip>/32  scram-sha-256
```

```
SELECT pg_reload_conf();      -- on db01, after editing pg_hba.conf
```

Confirm db02's `~postgres/.pgpass` holds the `replicator` password (mode 0600) and that `primary_conninfo` matches the working test string above.

C — “WAL segment has already been removed”

Hit during `pg_rewrite` (fetching pre-divergence WAL) or during catch-up (primary recycled WAL). Check the archive, enable archive fetch, and harden retention:

```
# Is the needed segment in the archive on db02's reachable path?
ls -l /apps/pgsql_archives/ | grep <segment_name>
```

```
# db02: let it pull missing WAL from the archive
restore_command = 'cp /apps/pgsql_archives/%f %p'
```

For `pg_rewrite`, add `--restore-target-wal` (already in step 2.3). On db01, prevent recurrence in future tests — the `ssncpri` slot already pins WAL while active; as an extra margin you can also set:

```
-- On db01
ALTER SYSTEM SET wal_keep_size = '8GB';
SELECT pg_reload_conf();
```

D — slot `ssncpri` will not drop on db01 (“is active”)

A walsender is still holding the slot. Identify and terminate it, then drop:

```
-- On db01
SELECT slot_name, active, active_pid
FROM pg_replication_slots WHERE slot_name = 'ssncpri';

-- Terminate the holding backend (only after confirming db02 should be disconnected)
SELECT pg_terminate_backend(active_pid)
FROM pg_replication_slots WHERE slot_name = 'ssncpri' AND active;

SELECT pg_drop_replication_slot('ssncpri');
```

4 Sign-off checklist

#	Verification	Done
D1	Pre-flight backup of db01 taken & verified	☐
D1	db02 confirmed caught up before promotion	☐
D1	db02 promoted — <code>pg_is_in_recovery()</code> =f , timeline +1	☐
D1	db02 slot dropped on db01 (or WAL space being monitored)	☐
D1	App team handed off to db02; db01 still serving production	☐
D2	Fresh backup of db01 taken before rebuild	☐
D2	Slot recreated on db01; <code>wal_level</code> / <code>max_wal_senders</code> / <code>archive_mode</code> confirmed	☐
D2	db02 cleanly shut down (<i>state: shut down</i>)	☐
D2	Rebuild completed (<code>pg_rewind</code> or <code>pg_basebackup</code>)	☐
D2	<code>primary_conninfo</code> + <code>primary_slot_name</code> present; <code>standby.signal</code> exists	☐
D2	db02 read-only confirmed (write rejected)	☐
D2	<code>pg_stat_replication</code> on db01 shows db02 streaming; slot active	☐
D2	Replay lag ~0; timelines match; sentinel row replicated	☐
D2	Old db02 <code>PGDATA</code> copy removed only after validation passed	☐

5 To confirm before execution

Replication role (`replicator`), slot (`ssncpri`) and the default binary path (`/usr/pgsql-16/bin`) are locked in. These remain open — confirm them and the runbook is fully execution-ready:

1. Full **primary FQDN** — the image shows `ars-acm-nibc-...-db01.ssnc-corp.cloud` (the segment after `nibc-` is cut off; standby is `...-ams-postgres-db02`).

2. **pg_rewind viability:** is `wal_log_hints = on` or are data checksums enabled on db02? This decides whether Option A is possible. Quick check: `pg_controldata /apps/pgsql_data/16 | grep -i checksum` and `SHOW wal_log_hints;`.
3. **Replication auth method** (scram / cert / trust) — affects `.pgpass` and the rewind-role setup.
4. **Listen port** (assumed `5432`) and the **pgBackRest stanza** name for the safety backups.
5. Confirm production stays live on db01 during the test (this runbook assumes yes → no primary downtime).